Integração de um acelerador ISFFT ao processador NEORV32 (RISC-V): verificação UVM, DFT, prototipagem em FPGA e fluxo ASIC até o GDSII em SkyWater 130 nm

Lukas Araújo da Silva
Engenharia Eletrônica
Universidade de Brasília (UnB)
Brasília, Brasil
lukas.araujo.cp@gmail.com

***Abstract*—Este relatório descreve a etapa final do projeto do Programa CI Digital (MCTI/Softex). O módulo *Inverse Symplectic Finite Fourier Transform* (ISFFT), desenvolvido no Estudo Dirigido 1 para a modulação *Orthogonal Time Frequency Space* (OTFS) em enlaces com satélites LEO, foi integrado como periférico do processador RISC-V NEORV32. O IP foi verificado com a metodologia UVM no Cadence Xcelium: 10 testes aprovados, 9.928 palavras comparadas bit a bit, 100% de cobertura funcional e 8 de 8 asserções. Recebeu também cadeias de varredura (*scan*) e padrões de ATPG gerados no Cadence Modus, com 99,65% de cobertura de falhas *stuck-at* em 170 padrões e um custo de 17,2% em área de lógica. O sistema NEORV32 + ISFFT foi prototipado na Kria KR260 a 100 MHz, onde o processador RISC-V controla sozinho o DMA e o acelerador, com latência de 6,87 μs por matriz e EVM idêntico ao modelo de referência (3,41/0,78/0,60%). Para o fluxo ASIC, a arquitetura foi adaptada com memórias geradas pela OpenRAM e sintetizada no Cadence Genus em SkyWater 130 nm: 1,91 mm^2, folga de +2,41 ns a 50 MHz, equivalência formal confirmada no Conformal e simulação pós-síntese idêntica ao RTL. Como atividade extra, o netlist foi levado ao GDSII com ferramentas abertas (LibreLane/OpenROAD), incluindo o anel de *pads*, com DRC e LVS limpos.**

***Index Terms*—OTFS, ISFFT, RISC-V, NEORV32, UVM, DFT, ATPG, FPGA, Kria KR260, ASIC, SkyWater 130 nm, OpenRAM, LibreLane.**

I. Introdução

A. O Efeito Doppler em Enlaces com Satélites LEO

Constelações de satélites em órbita baixa (LEO) oferecem conectividade com baixa latência em qualquer ponto do planeta, mas o satélite se desloca a cerca de 7,5 km/s em relação à estação terrena. Como ilustra a Fig. 1, o sinal recebido sofre um desvio de frequência Δf que muda de sinal durante a passagem:

$$\Delta f = \frac{v}{c} f_0, \tag{1}$$

o que resulta em cerca de ± 50 kHz para uma portadora de 2 GHz e ± 300 kHz em 12 GHz (banda Ku). Esses valores superam o espaçamento de 15 kHz entre subportadoras do OFDM usado no LTE e no 5G. A modulação OFDM perde então a ortogonalidade entre subportadoras, surge Interferência Inter-Canal (ICI) e o canal varia dentro de um mesmo quadro, o que exige equalização complexa e muitos pilotos [1] [2].

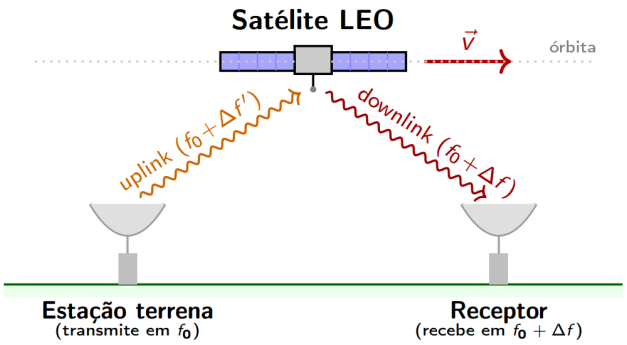


Fig. 1. Desvio Doppler em um enlace com satélite LEO: a frequência recebida $f_0 + \Delta f$ depende da velocidade relativa entre o satélite e a estação.

B. O Domínio Atraso-Doppler

A modulação OTFS organiza os símbolos de informação em uma grade bidimensional no domínio atraso-Doppler, cujos eixos representam o tempo de chegada (τ) e a velocidade relativa (ν) de cada refletor (Fig. 2). Nesse domínio, cada caminho de propagação é um ponto: o canal é esparso e permanece quase constante durante o quadro. Cada símbolo se espalha por todo o quadro tempo-frequência e experimenta a diversidade completa do canal. Assim, o OTFS não elimina o Doppler: ele o representa de forma que o receptor consiga separá-lo [1] [3].

C. A ISFFT no Transmissor OTFS

A ISFFT converte a grade de símbolos $X_{dd}[k, l]$ do domínio atraso-Doppler para a grade tempo-frequência $X_{tf}[n, m]$, que segue para a transformada de Heisenberg, equivalente a um modulador OFDM (Fig. 3). Para uma matriz $N \times M$ (8×8 neste trabalho) [3]:

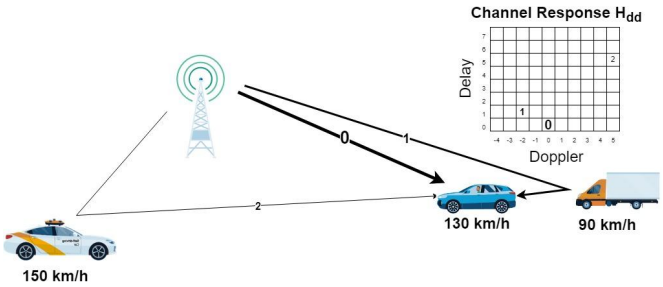


Fig. 2. Representação física do canal no domínio atraso-Doppler, adaptado de [2].

$$X[n,m] = \frac{1}{\sqrt{NM}} \sum_{k=0}^{N-1} \sum_{l=0}^{M-1} X[k,l]\, e^{j2\pi\left(\frac{nk}{N} - \frac{ml}{M}\right)} \qquad (2)$$

A transformada é separável: equivale a uma IFFT de N pontos ao longo de um eixo seguida de uma FFT de M pontos ao longo do outro. Essa propriedade define a arquitetura em hardware, com dois núcleos unidimensionais e uma memória de transposição entre eles.

D. Objetivos

O objetivo desta etapa é implementar o IP ISFFT desenvolvido no Estudo Dirigido 1 [14] como periférico do processador NEORV32 [5], validando o IP com UVM e DFT, o sistema na FPGA Kria KR260 e o fluxo ASIC até o GDSII. Os objetivos específicos são:

- Verificar o IP ISFFT com um ambiente UVM, com relatório de funcionalidade e de cobertura.
- Inserir cadeias de *scan* no IP e gerar padrões de teste por ATPG.
- Integrar o IP ao NEORV32 na Kria KR260, com o processador RISC-V controlando o acelerador e o DMA.
- Adaptar o sistema para ASIC em SkyWater 130 nm e realizar a síntese, a simulação pós-síntese e a verificação de equivalência no Cadence.
- Como atividade extra, levar o netlist sintetizado até o GDSII com ferramentas abertas.

II. Arquitetura e Metodologia

A. Fluxos de Desenvolvimento e Ferramentas

O trabalho seguiu dois fluxos. No **fluxo embarcado**, o RTL do NEORV32 (VHDL) e da ISFFT (Verilog) foi integrado no Vivado, o firmware do RISC-V foi compilado com o *riscv-none-elf-gcc* e a validação foi feita por *hardware-in-the-loop* com o PYNQ, como no Estudo Dirigido 1. No **fluxo ASIC**, a parte 1 usa o Cadence (síntese, simulação pós-síntese, equivalência, UVM e DFT) e a parte 2, extra, usa ferramentas abertas para o layout. A divisão é necessária porque o ambiente Cadence disponível não tem as regras físicas do SkyWater 130 nm para DRC, LVS e extração. A Tabela I resume as ferramentas.

Tabela I
Ferramentas utilizadas em cada etapa

Etapa	Ferramenta (versão)
Prototipagem em FPGA	Vivado, PYNQ (Kria KR260)
Síntese lógica	Cadence Genus 22.16
Simulação RTL e pós-síntese	Cadence Xcelium 23.09
Verificação UVM e cobertura	Cadence Xcelium 26.03, IMC 26.03
Equivalência formal (LEC)	Cadence Conformal 23.2 e 26.1
ATPG	Cadence Modus 25.1
Macros de memória	OpenRAM 1.2.49
NEORV32: VHDL → Verilog	GHDL
Layout (extra)	LibreLane 3.0.14 (OpenROAD, Magic, Netgen, KLayout)

B. O IP ISFFT

O núcleo de cálculo é o mesmo do Estudo Dirigido 1; a Fig. 4 mostra o caminho de dados do IP na versão ASIC. Dois núcleos de 8 pontos gerados pelo Spiral [4], com representação em ponto fixo de 16 bits para as partes real e imaginária, latência de 41 ciclos e uma transformada a cada 4 ciclos, realizam a IFFT nas linhas e a FFT nas colunas. Entre eles, um banco *ping-pong* de $2 \times 8 \times 8$ palavras de 32 bits faz a transposição: uma matriz é escrita por linhas em um banco enquanto a anterior é lida por colunas do outro. O modelo de referência em Python aplica *ifft* nas linhas, *fft* nas colunas e divide o resultado por 8.

Para ser controlado pelo RISC-V, o núcleo recebeu um invólucro (*wrapper*) com três interfaces: AXI-Stream de entrada e de saída para os dados e AXI4-Lite para o controle. Cada palavra de 64 bits carrega duas amostras complexas, portanto uma matriz 8×8 ocupa 32 palavras. O *wrapper* contém:

- **Buffer de entrada** de 64 palavras com *commit*/*rollback*: a matriz só é confirmada após 32 palavras; um `tlast` prematuro descarta a matriz truncada sem travar o núcleo.
- **Liberador por crédito**: só entrega uma matriz ao núcleo se houver espaço reservado na saída, de modo que nenhum resultado seja perdido.
- **FIFO de saída** de 256 palavras, que respeita o `tready` (contrapressão) e gera o `tlast` a cada `PKT_MAT` matrizes.

Os registradores AXI4-Lite estão na Tabela II. Na versão ASIC (`isfft_axi_v4`), o buffer de entrada e a FIFO de saída passaram a ser macros SRAM, o que exigiu uma pré-busca, porque a macro só oferece leitura síncrona.

Tabela II
Mapa de registradores AXI4-Lite do IP

Offset	Registrador	Função
0x00	CTRL (R/W)	*enable*, *soft reset*, limpeza de erros
0x04	STATUS (RO)	ocupado, FIFO cheia/com dados, erros
0x08	MAT_IN (RO)	matrizes confirmadas na entrada
0x0C	MAT_OUT (RO)	matrizes entregues na saída
0x10	ERR_CNT (RO)	erros de `tlast`
0x14	ID (RO)	identificação `0x15FF0300`
0x18	PKT_MAT (R/W)	matrizes por pacote de saída

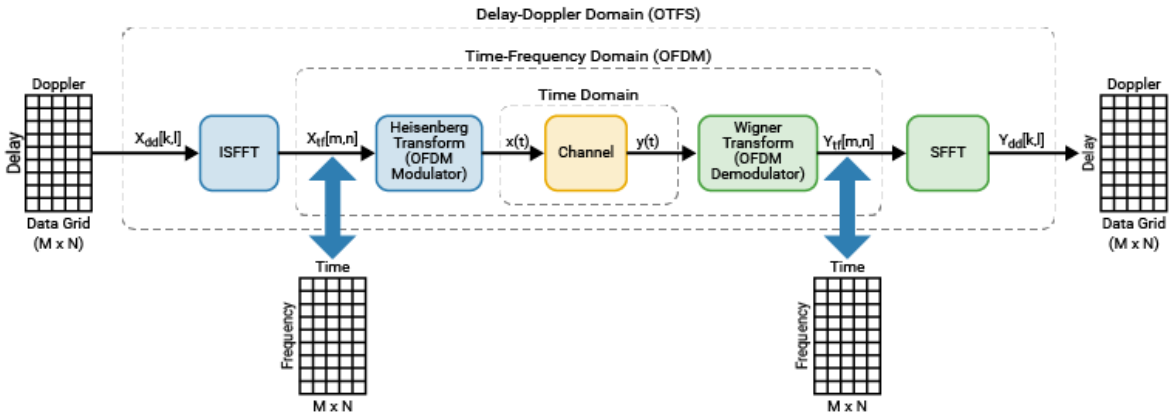


Fig. 3. Cadeia de modulação OTFS com os domínios atraso-Doppler e tempo-frequência, adaptado de [2]. A ISFFT é o pré-processador do transmissor.

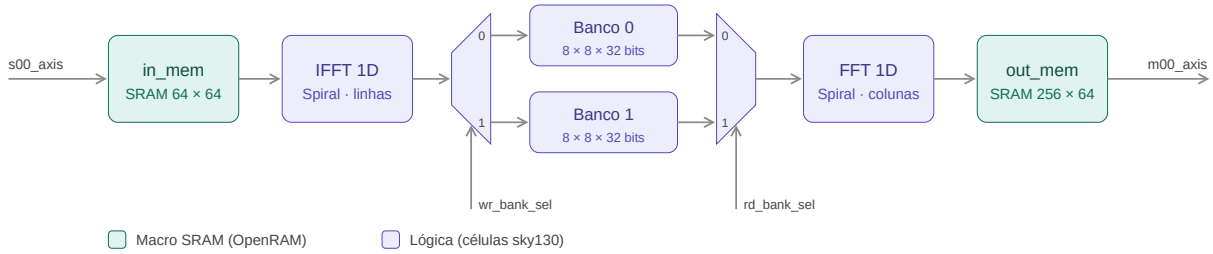


Fig. 4. Bloco operacional do IP ISFFT na versão ASIC: buffer de entrada (`in_mem`) e FIFO de saída (`out_mem`) em macros OpenRAM, IFFT 1D, banco *ping-pong* de transposição selecionado por `wr_bank_sel`/`rd_bank_sel` e FFT 1D.

C. Arquitetura com o NEORV32 na Kria KR260

Na Kria (Fig. 5), o NEORV32 é o controlador do SoC na lógica programável (PL). Ele configura a ISFFT pelo AXI4-Lite e programa um AXI DMA que lê a matriz da memória DDR, a envia pelo AXI-Stream à ISFFT e grava o resultado de volta. O firmware e uma caixa de correio ficam em uma BRAM de 64 kB. O sistema de processamento (PS), com o ARM Cortex-A53 rodando Ubuntu e PYNQ, apenas carrega o *bitstream* e o firmware, aloca os buffers na DDR e serve como depurador. O *block design* correspondente está na Fig. 6.

D. Arquitetura com o NEORV32 no ASIC

Em ASIC não existem o Zynq, o AXI DMA e os demais IPs da Xilinx. A arquitetura foi adaptada como mostra a Fig. 7:

- O NEORV32 (rv32imc) é o único processador; o programa fica na IMEM, carregada pelo *testbench*.
- O DMA e o SLINK do próprio NEORV32 movem os dados. A FIFO do SLINK tem 64 posições, o suficiente para uma matriz inteira, e conversores de 32 para 64 bits ligam o SLINK à ISFFT.
- Uma ponte XBUS → AXI4-Lite dá acesso aos registradores do IP.
- O NEORV32 foi convertido de VHDL para um único Verilog com o GHDL, porque o fluxo aberto não lê VHDL.
- O relógio passou de 100 para 50 MHz, e o banco de registradores da CPU foi implementado em *flip-flops*.

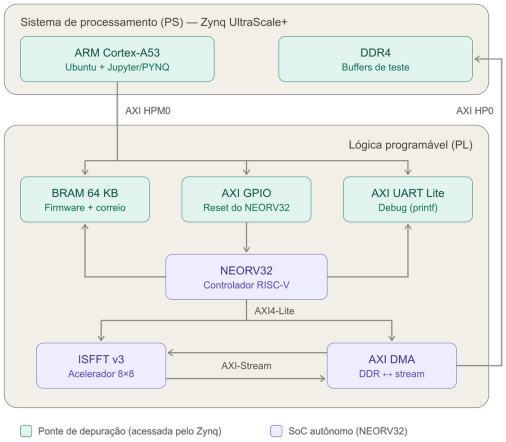


Fig. 5. Arquitetura na Kria KR260: o NEORV32 controla a ISFFT e o AXI DMA; o Zynq atua como ponte de depuração.

E. Tecnologia: SkyWater 130 nm e OpenRAM

O PDK SkyWater 130 nm [6] é aberto (licença Apache 2.0) e traz tudo de que o fluxo aberto precisa para chegar ao GDSII: LEF, Liberty, GDS e as regras do Magic, do Netgen e do KLayout. Não exige NDA, de modo que layout, macros e documentação podem ser publicados. Foi usada a biblioteca de alta densidade `sky130_fd_sc_hd` a 1,8 V, com cinco camadas de metal (met1 a met5) além da camada *li1*.

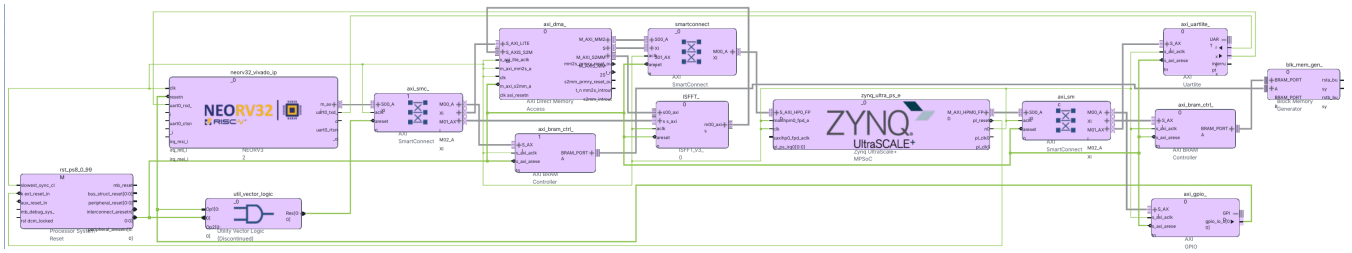


Fig. 6. *Block design* no Vivado com o NEORV32, a ISFFT, o AXI DMA, as BRAMs e o Zynq UltraScale+.

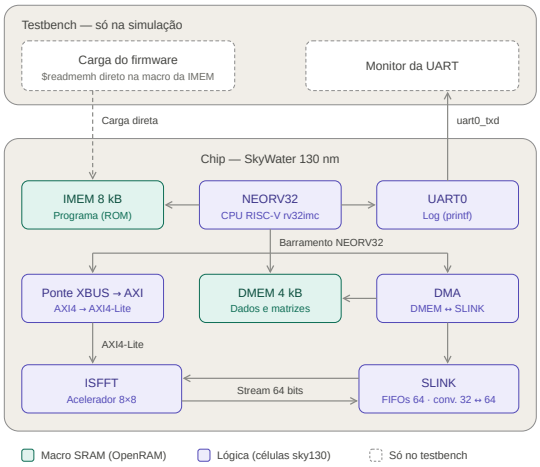


Fig. 7. Arquitetura do SoC em ASIC: IMEM e DMEM em macros OpenRAM, transporte de dados por DMA e SLINK do NEORV32.

O PDK aberto não inclui compilador de memória da *foundry*. As memórias grandes foram geradas pela OpenRAM [7], um compilador aberto que, a partir de um arquivo de configuração (largura da palavra, profundidade, portas, escrita por byte), produz a macro completa: GDS, LEF, Liberty, Verilog e SPICE (Fig. 8). Uma memória em *flip-flops* custa cerca de 43,9 μm^2/bit, contra 8,7 μm^2/bit na macro de 4 kB. Uma macro de duas portas (1rw1r) de 64 bits, porém, tem um custo fixo de cerca de 103.900 μm^2 mais 11,8 μm^2/bit, e por isso só compensa acima de aproximadamente 3.200 bits. Essa regra, medida com macros de vários tamanhos, definiu a implementação de cada memória (Tabela III): quatro macros no total. O número de macros é ditado também pelos acessos por ciclo, e não só pelos bits: o *ping-pong* faz duas leituras e duas escritas por ciclo.

F. Estratégia de Validação: as Três Matrizes de Teste

Desde o Estudo Dirigido 1, as mesmas três matrizes de entrada são usadas em todas as etapas: um impulso na posição [3,3] com amplitude 1000, uma senoide 2D com amplitude 500 e uma rampa com valor $10\times$ índice (Fig. 9). A precisão é medida pelo *Error Vector Magnitude* contra o modelo em ponto flutuante:

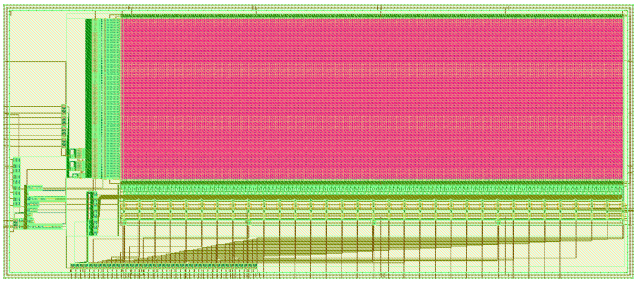


Fig. 8. Layout da macro OpenRAM de 4 kB (DMEM), $808{,}8 \times 351{,}3$ μm: matriz de *bitcells* e periferia gerada pela OpenRAM.

Tabela III
IMPLEMENTAÇÃO DAS MEMÓRIAS DO SOC

Memória	Tamanho	Implementação	Área (μm^2)
IMEM (programa)	2048×32	Macro 1rw 8 kB	448.827
DMEM (dados)	1024×32	Macro 1rw 4 kB	284.139
FIFO de saída da ISFFT	256×64	Macro 1rw1r	296.939
Buffer de entrada da ISFFT	64×64	Macro 1rw1r	152.932
Ping-pong da ISFFT	$2 \times 8 \times 8 \times 32$	*Flip-flops*	–
Memórias internas do Spiral	$16 \times (4$ ou $8) \times 32$	*Flip-flops*	–
FIFOs do SLINK, DMA e UART	$\leq 64 \times 37$	*Flip-flops*	–

$$\text{EVM} = \sqrt{\frac{\text{média}\,|X_{hw} - X_{ref}|^2}{\text{média}\,|X_{ref}|^2}} \times 100\% \qquad (3)$$

No sistema ASIC, o firmware do NEORV32 calcula também uma soma de verificação de cada uma das três saídas. Usar sempre o mesmo estímulo torna qualquer divergência visível de imediato. A Tabela IV resume onde esse estímulo foi aplicado ao longo do projeto.

Tabela IV
AS TRÊS MATRIZES AO LONGO DO PROJETO

Etapa	Resultado
Kria KR260 (placa)	EVM 3,4122 / 0,7822 / 0,5965%
Simulação RTL do SoC	EVM idêntico à placa
UVM (teste `kria`)	EVM idêntico à placa
Pós-síntese (Genus)	3 somas iguais ao RTL
Netlist do layout	3 somas iguais ao RTL
Chip com os *pads*	3 somas iguais ao RTL

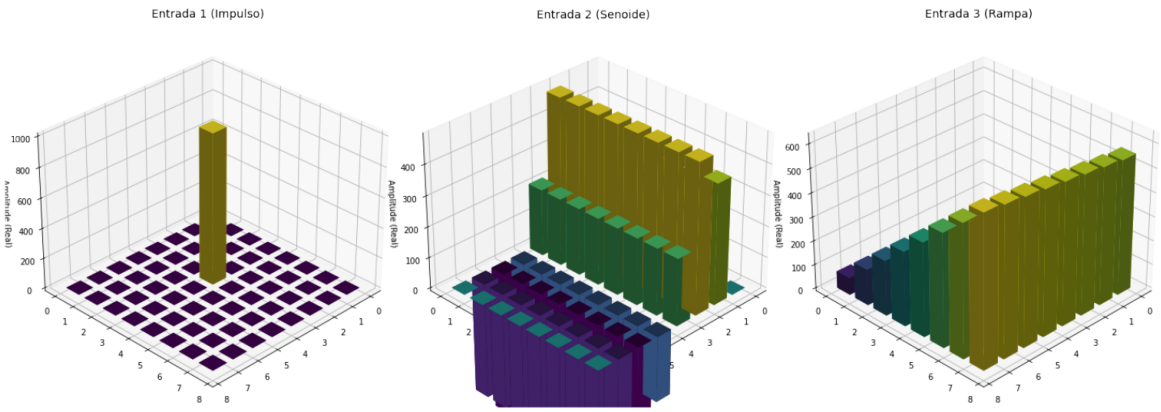


Fig. 9. Matrizes de entrada 8×8 usadas em todas as etapas: impulso, senoide e rampa.

III. Verificação do IP com UVM

A. Proposta do Ambiente

O ambiente UVM (Fig. 10) verifica apenas o IP, sem o processador. O DUT é o `isfft_axi_v4`, o mesmo RTL sintetizado no Cadence, com os modelos das duas macros SRAM. O ambiente é composto por:

- **Agentes** AXI4-Lite e AXI-Stream de entrada e de saída, com atrasos, pausas e contrapressão sorteados (`tready` entre 50 e 100% e paradas longas).
- **RAL** com os 7 registradores; os contadores são previstos pelo modelo e conferidos.
- **Modelo de referência** que reproduz o *commit/rollback* da entrada e passa cada matriz pelo *wrapper* validado na etapa 1.
- ***Scoreboard*** com duas checagens independentes: bit a bit contra o modelo de referência e numérica contra a ISFFT em ponto flutuante, com tolerância de 4 LSB, além de um modelo do `tlast`.
- **Oito asserções** ligadas ao DUT por `bind`, sem alterar o RTL, e checagem de protocolo AXI nas interfaces.
- **Cobertura funcional** em 4 grupos com 54 *bins*: entrada, saída, registradores e ocupação dos buffers.

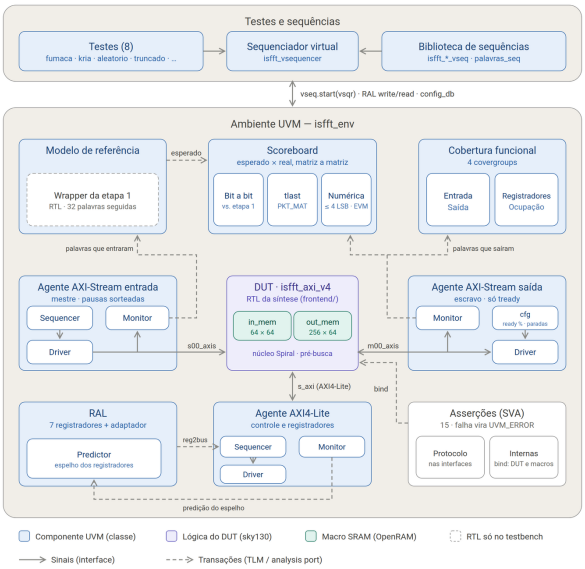


Fig. 10. Ambiente UVM do IP ISFFT: agentes, RAL, modelo de referência, *scoreboard*, asserções e cobertura.

A simulação usou o Xcelium 26.03 com UVM 1.1d [10] a 50 MHz. Antes de usar a máquina com as licenças, o ambiente foi conferido no Verilator 5.052 com 12 defeitos plantados no RTL: todos foram detectados. Os dois defeitos inseridos no núcleo Spiral só foram vistos pela checagem numérica, porque a referência bit a bit usa o mesmo núcleo; isso justifica manter as duas checagens.

B. Resultados de Funcionalidade

A regressão final tem 10 testes (Tabela V). Todos passaram, com 0 erros de dado, 0 erros de `tlast`, 0 `UVM_ERROR`, 0 `UVM_WARNING` e nenhuma asserção falhando. O pior erro numérico dentro da faixa válida foi de 3,14 LSB. O teste `kria` reproduziu exatamente o EVM medido na placa.

A verificação revelou um limite do núcleo que os testes da etapa 1 não haviam mostrado: o Spiral estoura quando $|\mathrm{Re}|$ ou $|\mathrm{Im}|$ da entrada passa de $32.767/\sqrt{2} \approx 23.170$, porque o estágio que multiplica pelo fator de 45° da IFFT não tem bit de guarda. A comparação bit a bit não acusa o problema, já que a referência tem o mesmo núcleo; só o modelo independente em ponto flutuante o detectou. A faixa válida da entrada foi documentada e o teste `estouro` passou a medi-la. Também se observou que o erro absoluto é de cerca de 3 LSB em qualquer amplitude, o que motivou usar um critério em LSB, e não em EVM, para sinais pequenos.

C. Resultados de Cobertura

A cobertura foi consolidada no Cadence IMC com o *merge* das 10 rodadas (Fig. 11 e Tabela VI). Os buracos encontrados na primeira medição viraram testes: uma espera curta na entrada, que só ocorre com entrada contínua e saída livre (teste `vazao`); truncamentos na primeira e na última palavra (pontas dirigidas); e um *soft reset* com o núcleo em operação. A cobertura de código do *wrapper* ficou em 73,5% por dois

Tabela V
REGRESSÃO UVM NO XCELIUM 26.03 (SEMENTE 1)

Teste	Cenário	Palavras bit a bit	Pior erro numérico	Resultado
`fumaca`	1 matriz	32	0,77 LSB	Passou
`registradores`	*reset*, leitura/escrita, escrita em registrador RO	–	–	Passou
`kria`	3 matrizes da placa (EVM igual ao da placa)	96	1,48 LSB	Passou
`aleatorio`	40 matrizes, 5 tipos × 4 amplitudes	1.280	3,14 LSB	Passou
`truncado`	pacotes truncados, com as pontas dirigidas	1.280	2,98 LSB	Passou
`contrapressao`	60 matrizes, `tready` 50% e paradas longas	1.920	3,14 LSB	Passou
`soft_reset`	2 *soft resets*, um com o núcleo calculando	200	2,48 LSB	Passou
`kria_regressao`	*testbench* da Kria reescrito em UVM	1.280	2,94 LSB	Passou
`vazao`	80 matrizes sem pausa, `tready` 100%	2.560	$\leq$ 4 LSB	Passou
`estouro`	40 matrizes acima da faixa válida (só contado)	1.280	40 de 40 estouram	Passou
Total	**10 testes**	**9.928**	**$\leq$ 3,14 LSB**	**10/10**

motivos: a métrica passou a contar expressões (`&`, `|`, `assign`), acrescentando 126 itens, e um bloco da FSM de escrita é inalcançável, pois o liberador gasta 33 ciclos por matriz e nunca emenda duas matrizes. Os modelos das macros e as asserções foram excluídos da cobertura de código, por não serem RTL do chip.

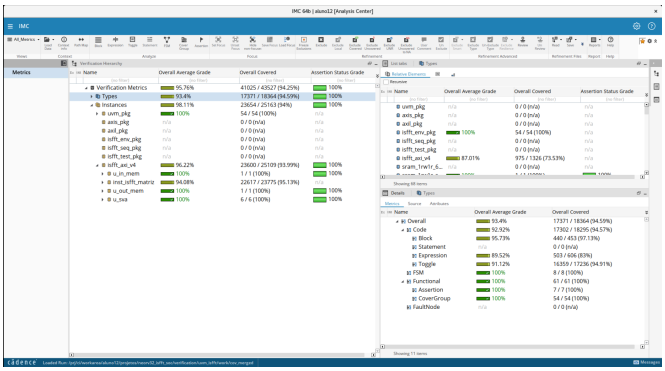


Fig. 11. Relatório de cobertura no Cadence IMC (*merge* das 10 rodadas).

Tabela VI
COBERTURA DO IP NO IMC

Métrica	1ª rodada (8 testes)	Final (10 testes)
Funcional (54 *bins*)	94,4%	**100%**
Asserções	8/8	**8/8**
Código do núcleo ISFFT	99,6%	**100%** (280/280)
Código do *wrapper*	70,7%	**73,5%** (975/1.326)

IV. FLUXO DFT NO IP ISFFT

A. Metodologia

O teste de fabricação foi aplicado apenas ao IP. Na técnica de *scan*, cada *flip-flop* recebe um multiplexador e passa a ser um elo de uma cadeia de deslocamento: o padrão é carregado pela cadeia (`scan_en` = 1), o circuito funciona por um ciclo (`scan_en` = 0) e a resposta é descarregada e comparada. Com todos os *flip-flops* acessíveis, a geração automática de padrões (ATPG) resolve um problema combinacional: para cada falha *stuck-at*, um pino preso em 0 ou em 1, encontra valores que a ativem e propaguem até um ponto observável.

O fluxo teve cinco passos:

1) Síntese com *scan* completo (*muxed scan*) no Genus, com 16 cadeias.
2) Equivalência entre o RTL e o netlist com *scan* no Conformal.
3) Simulação das três matrizes no netlist com `scan_en` = 0, para garantir que a inserção não alterou a função.
4) ATPG *stuck-at* no Modus [12].
5) Simulação no Xcelium dos padrões gerados aplicados ao netlist.

Não foi necessário um sinal de modo de teste, pois o IP não tem *reset* assíncrono, *clock gating* nem *latches*. As macros SRAM foram tratadas como caixa-preta, sem lógica de contorno.

B. Resultados

Os cinco passos foram aprovados (Tabela VII). Os 170 padrões atingiram 99,65% de cobertura *stuck-at* no modo FULLSCAN (Tabela VIII). As falhas restantes concentram-se em torno das macros: as 214 classificadas como possíveis ocorrem porque o padrão gera X na saída da caixa-preta, e as 151 redundantes estão em lógica sem efeito observável.

Tabela VII
RESULTADOS DE CADA PASSO DO FLUXO DFT

Passo	Resultado
Síntese com *scan* (Genus)	12.552 FF, 100% com *scan*, 16 cadeias de ∼785
LEC RTL × netlist (Conformal)	Aprovado, 0 não-equivalentes
Três matrizes com `scan_en`=0 (Xcelium)	Aprovado
ATPG *stuck-at* (Modus)	170 padrões, 99,65%
Padrões no netlist (Xcelium)	170/170, 0 divergências em 2.156.112 comparações

Tabela VIII
COBERTURA DE FALHAS *stuck-at* NO MODUS

Modo	Falhas	Testadas	Possíveis	Redund.	Não test.	Cobertura
FULLSCAN	341.871	340.510	214	151	996	99,65%
Global	343.063	340.510	214	151	2.188	99,30%

O custo da testabilidade está na Tabela IX. A área de lógica cresce 17,2%, porque cada *flip-flop* é substituído por uma célula com o multiplexador de *scan* embutido; com as duas macros, o aumento total é de 8,6%. O IP sem *scan* usado na comparação é o da síntese do SoC, com a mesma biblioteca e o mesmo RTL. A folga de *setup* não é comparável entre as duas sínteses, pois as restrições do IP isolado são outras.

Tabela IX
CUSTO DO *scan* EM ÁREA

	Sem *scan*	**Com *scan***	**Diferença**
Área total com 2 macros (μm^2)	897.901	974.849	+8,6%
Somente lógica (μm^2)	448.030	524.977	+17,2%
Instâncias	36.375	35.962	−413

V. PROTOTIPAGEM EM FPGA

A. Utilização de Recursos, Temporização e Potência

O sistema completo foi implementado na Kria KR260 [13] (Zynq UltraScale+ *xck26-sfvc784-2LV-c*) a 100 MHz (Fig. 12). Todas as restrições de temporização foram atendidas, com WNS de +3,903 ns e WHS de +0,010 ns, sem caminhos violando entre 49.570 pontos de chegada. A utilização está na Tabela X. Em relação ao Estudo Dirigido 1, o aumento vem do NEORV32 e da BRAM de 64 kB; a ISFFT sozinha usa 9,5 BRAMs.

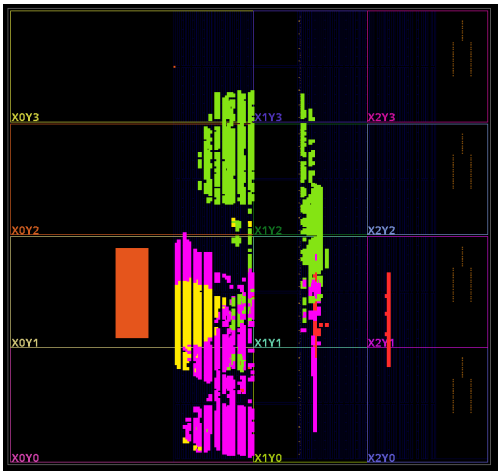


Fig. 12. Posicionamento do sistema NEORV32 + ISFFT no dispositivo (Vivado).

Tabela X
UTILIZAÇÃO DE RECURSOS NA KRIA KR260

Recurso	**Utilização**	**Disponível**	**Utilização (%)**
LUT	10.376	117.120	8,86
LUTRAM	1.301	57.600	2,26
FF	15.080	234.240	6,44
BRAM	36,5	144	25,35
DSP	16	1.248	1,28

A potência total estimada pelo Vivado foi de 2,846 W, dos quais 2,545 W são dinâmicos e 0,301 W estáticos. O PS consome 2,417 W (94% da potência dinâmica), de modo que a lógica programável, incluindo o NEORV32, a ISFFT e o DMA, responde por cerca de 0,128 W de potência dinâmica.

B. Validação Hardware-in-the-Loop

O notebook Jupyter grava o firmware na BRAM, escreve na caixa de correio os endereços dos buffers e libera o NEORV32. O processador programa o DMA e a ISFFT, mede os ciclos com o contador de ciclos do RISC-V e devolve o status. A saída do hardware (Fig. 13) foi comparada com o modelo de referência (Tabela XI): os valores de EVM são idênticos aos do Estudo Dirigido 1, agora com o RISC-V no controle. A latência do hardware foi de 506 ciclos (5,06 μs) por matriz e, incluindo a programação do DMA pelo NEORV32, de 687 ciclos (6,87 μs).

Tabela XI
RESULTADOS *hardware-in-the-loop* A 100 MHZ

Matriz	**EVM (%)**	**MAE**	**Ciclos**	**Com DMA**
1 (Impulso)	3,4122	0,40	506	687
2 (Senoide)	0,7822	0,10	507	687
3 (Rampa)	0,5965	0,09	506	687

A latência por matriz é maior que os 320 ns do Estudo Dirigido 1 porque as métricas diferem: lá se mediu a vazão do *pipeline* com várias matrizes em sequência; aqui, o tempo de ida e volta de uma matriz isolada pelo DMA, com o RISC-V programando cada transferência. Comparado ao tempo médio de 0,81 ms do modelo em Python no ARM medido no Estudo Dirigido 1 [14], o sistema com o NEORV32 é cerca de 118 vezes mais rápido. Em energia por matriz ($E = P \times t$), a estimativa é de 0,128 W $\times$ 6,87 μs $\approx$ 0,88 μJ para o caminho em hardware, contra 2,417 W $\times$ 0,81 ms $\approx$ 1,96 mJ para o ARM. Esses valores partem das estimativas de potência do Vivado, e não de medições na placa, e por isso devem ser lidos como ordem de grandeza.

VI. FLUXO ASIC NO CADENCE

A. Síntese Lógica

O SoC foi sintetizado no Genus [11] com a biblioteca `sky130_fd_sc_hd` no canto típico (TT, 25 °C, 1,80 V), os arquivos Liberty das quatro macros e relógio de 20 ns (50 MHz). A síntese levou cerca de 10 minutos e resultou em 1,913 mm^2, 60.026 instâncias (19.823 *flip-flops* e 4 macros), folga de *setup* de +2,41 ns e nenhuma violação de transição, capacitância ou *fanout*. A distribuição de área está na Tabela XII.

As memórias dominam o chip: somando as macros com as memórias que ficaram em *flip-flops* (FIFOs do SLINK, *ping-pong* e banco de registradores), quase 80% da área é memória, enquanto a aritmética da FFT e da IFFT ocupa 14,6%. Essa assinatura aparece também nas células: 89% dos *flip-flops* não têm *reset* e 62% das células combinacionais são `a22o`/`a22oi`, o multiplexador de leitura dessas memórias.

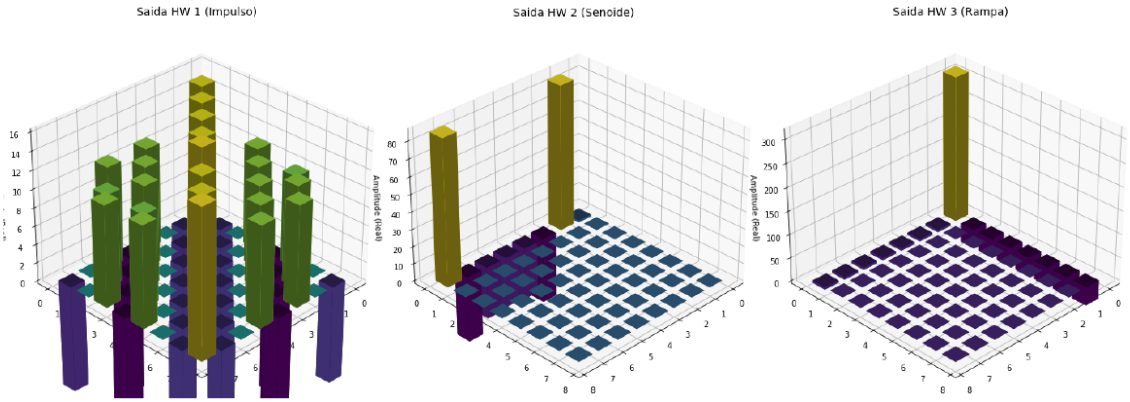


Fig. 13. Saída do hardware na Kria (domínio tempo-frequência) para as três matrizes de teste, capturada pelo notebook PYNQ.

Tabela XII
DISTRIBUIÇÃO DE ÁREA NA SÍNTESE DO SOC

Bloco	**Área (μm^2)**	**% do chip**
Macros SRAM (4)	1.182.838	61,8
Lógica da ISFFT	448.030	23,4
ping-pong e registradores	149.015	7,8
IFFT 1D / FFT 1D (Spiral)	141.104 / 138.334	7,4 / 7,2
wrapper `isfft_axi_v4`	18.892	1,0
Lógica do NEORV32	275.798	14,4
FIFOs do SLINK ($2 \times 64 \times 37$)	147.333	7,7
CPU rv32imc	97.058	5,1
Conversores $32 \leftrightarrow 64$	6.205	0,3
Total	**1.912.952**	**100**

Tabela XIII
VERIFICAÇÃO DA SÍNTESE

Verificação	**Pergunta**	**Resposta**
Genus	Fecha temporização?	Sim: +2,41 ns a 50 MHz
Conformal LEC	Netlist $\equiv$ RTL?	Sim: 169/169 módulos
Xcelium pós-síntese	O netlist funciona?	Sim: 3 somas = RTL
Simulação RTL	A matemática está certa?	Sim: EVM = placa

O caminho crítico está na CPU, e não na ISFFT: uma subtração de 32 bits em *ripple carry* no multiplicador/divisor, com 32 somadores completos em cadeia que respondem por 12,5 ns dos 15,5 ns de atraso. Os dez piores caminhos estão todos na CPU, com cerca de 2,4 ns de folga.

B. Simulação Pós-Síntese e Equivalência

O netlist do Genus foi simulado no Xcelium com o mesmo firmware do RTL, processando as três matrizes por DMA e SLINK. Foi necessário compilar o modelo das células com as definições `FUNCTIONAL` e `UNIT_DELAY`, porque o modelo Verilog distribuído com o PDK aberto não tem blocos `specify`; sem elas, toda a simulação ficava em X. O resultado está na Tabela XIII: as três somas de verificação são iguais às do RTL (`0xeb88508e`, `0xba7fc58e` e `0xb29c0211`), com 188.195 ciclos no netlist contra 188.194 no RTL. Cada matriz leva 2.622 ciclos (52,4 μs a 50 MHz), incluindo a movimentação pelo DMA e pelo SLINK.

A equivalência formal no Conformal confirmou que o netlist implementa o mesmo circuito do RTL: 19.828 pontos equivalentes (19.823 *flip-flops*, 1 saída e 4 macros), nenhum não-equivalente ou abortado, e 169 de 169 módulos aprovados.

C. Observação sobre a Potência

O relatório de potência do Genus não pôde ser usado. O arquivo Liberty gerado pela OpenRAM declara potência interna fora de escala, e o total estimado chega a 2.767 W. Considerando só as células padrão, a estimativa é de cerca de 50 mW, obtida sem atividade de simulação (*vectorless*) e, portanto, válida apenas como ordem de grandeza. Uma estimativa confiável exige corrigir a potência das macros e usar a atividade da simulação pós-síntese.

VII. DO NETLIST AO GDSII COM FERRAMENTAS ABERTAS (ATIVIDADE EXTRA)

A. Layout do Núcleo

O netlist do Genus foi levado ao layout com o LibreLane [9], que encadeia o OpenROAD [8] (*floorplan*, rede de alimentação, posicionamento, árvore de relógio, roteamento e análise temporal), o Magic e o KLayout (GDS, DRC e comparação XOR) e o Netgen (LVS). O núcleo (`asic_top`) tem área de $2{,}0 \times 1{,}5$ mm com as quatro macros; as etapas estão na Fig. 14. Três problemas se destacaram:

- **Congestionamento:** o excesso de demanda no roteamento global caiu de 4.682 para 0 com densidade alvo de 65% e a DMEM movida para o canto, o que abriu o canal entre as macros.
- **Macros vazias no GDS:** a comparação XOR entre o Magic e o KLayout apontou 15,7 milhões de diferenças. O LEF gerado pela OpenRAM não tinha a declaração `FOREIGN`, e o KLayout gravava as macros vazias; com o LEF corrigido, as diferenças foram a zero.
- **Unidades do LEF:** o LEF da OpenRAM usa 2000 unidades por mícron, enquanto o OpenROAD só aceita 1000 no sky130, o que exigiu uma conversão.

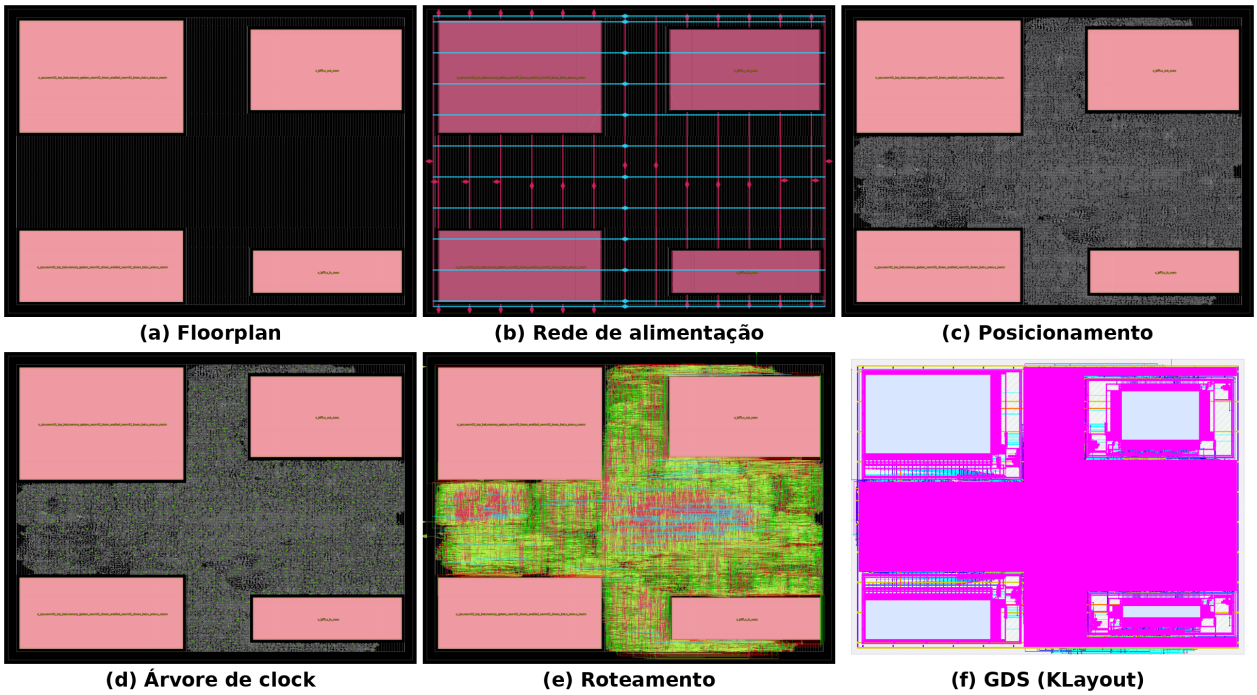


Fig. 14. Etapas do layout do núcleo no LibreLane: (a) *floorplan* com as macros, (b) rede de alimentação, (c) posicionamento, (d) árvore de relógio, (e) roteamento e (f) GDS final no KLayout.

B. Chip com o Anel de Pads

No segundo nível, o núcleo entrou como macro dentro de um anel de *pads* `sky130_ef_io`, em uma área de 2,70×2,20 mm (Fig. 15). São 11 *pads*: 4 de sinal (`clk`, `rstn`, `rx` e `tx`), 6 de alimentação e 1 fatia, além de 4 cantos. A primeira versão apresentou transição de 3,54 ns no *pad* `tx`, acima do limite de 1,5 ns. Com o modelo de tempo do núcleo disponível no nível do chip, o reparo do OpenROAD inseriu um *buffer* antes do *pad*, e a transição caiu para 0,11 ns.

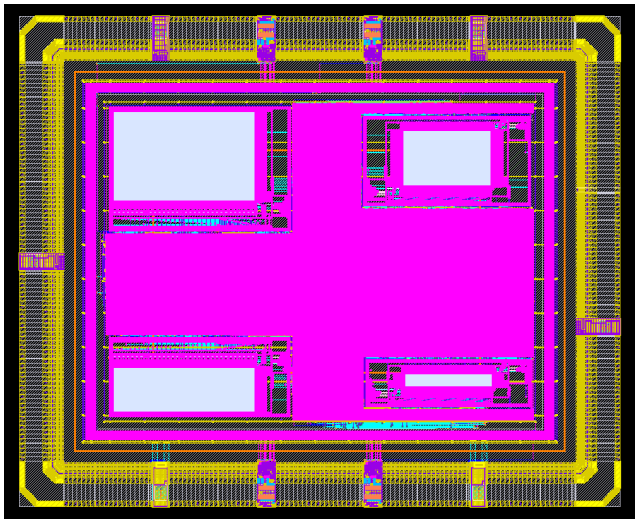


Fig. 15. Chip final com o anel de *pads* e o preenchimento de metal.

C. Verificação Final

A Tabela XIV resume a verificação nos dois níveis. No núcleo, os 130 erros de DRC restantes estão em pinos sem uso das macros, tratadas como caixa-preta. No chip, a faixa dos *pads* foi isenta, porque os erros apontados estão nas próprias células de E/S da *foundry*. A equivalência entre o netlist do layout e o do Genus e a simulação com o firmware fecham a cadeia: RTL ≡ Genus ≡ layout (LEC) ≡ GDS (LVS), e o desenho executa o firmware corretamente.

VIII. Conclusão e Próximos Passos

Este trabalho integrou o acelerador ISFFT ao processador RISC-V NEORV32 e o validou em três níveis. No nível do **bloco**, o IP foi verificado com UVM (10 de 10 testes, 100% de cobertura funcional, 8 de 8 asserções), e a verificação revelou e documentou um limite de estouro do núcleo que os testes anteriores não haviam mostrado. O IP também se tornou testável, com 99,65% de cobertura *stuck-at* ao custo de 17,2% de área de lógica. No nível do **sistema**, o NEORV32 controla sozinho o DMA e a ISFFT na Kria KR260, com latência de 6,87 μs por matriz e o mesmo EVM do modelo de referência. No nível do **silício**, o SoC foi sintetizado em SkyWater 130 nm com 1,91 mm^2 a 50 MHz, com equivalência formal e simulação pós-síntese idêntica ao RTL, e foi levado até o GDSII com o anel de *pads*, com DRC e LVS limpos.

O fio condutor do projeto foi usar sempre o mesmo estímulo: o EVM de 3,41/0,78/0,60% se repetiu na placa, na simulação RTL do SoC e no UVM, e as mesmas três somas de verificação se repetiram do RTL ao netlist do layout. Os próximos passos, no Trabalho de Conclusão de Curso, são:

Tabela XIV
VERIFICAÇÃO FINAL DO LAYOUT

Verificação	**Núcleo (`asic_top`)**	**Chip com o anel de *pads***
DRC (Magic)	0 erros reais (130 isentos: pinos sem uso das macros)	0 no desenho (faixa dos *pads* isenta)
LVS (Netgen)	*Circuits match uniquely*	*Circuits match uniquely*
XOR Magic × KLayout	0	0
Antena	0	0
LEC: netlist do layout × Genus	19.828 pontos equivalentes	–
Simulação com o firmware	3 somas = RTL	3 somas = RTL
STA *setup* (50 MHz)	+0,32 ns (TT)	+2,95 ns (TT); +2,37 ns no pior caso (SS)
STA *hold*	+0,13 ns (TT); 15 caminhos até −28 ps no canto FF (aceitos)	+3,63 ns

- **Transceptor completo:** substituir a cadeia ISFFT + Heisenberg pela Transformada Inversa de Zak no transmissor e implementar o receptor.
- **Escalabilidade:** crescer a matriz rumo a 1024×256, compatível com enlaces LEO, respeitando a faixa válida de entrada identificada pelo UVM.
- **Protótipo real:** migrar para a ZCU102 com o AD-FMCOMMS3 (AD9361) e realizar transmissão *over-the-air* entre duas placas.

REFERÊNCIAS

[1] R. Hadani et al., "Orthogonal Time Frequency Space Modulation," in IEEE WCNC, 2017.
[2] MathWorks, "OTFS Modulation," Documentation. [Online]. Disponível: https://www.mathworks.com/help/comm/ug/otfs-modulation.html.
[3] E. Viterbo, "OTFS Modulation," Monash University, 2019. [Online]. Disponível: https://users.monash.edu/~gfarr/research/slides/Viterbo_Nov_2019___OTFS_modulation.pdf.
[4] Spiral, "DFT Hardware Generator," [Online]. Disponível: https://www.spiral.net/hardware/dftgen.html.
[5] S. Nolting, "The NEORV32 RISC-V Processor," [Online]. Disponível: https://github.com/stnolting/neorv32.
[6] Google e SkyWater Technology, "SkyWater SKY130 Open Source PDK," [Online]. Disponível: https://github.com/google/skywater-pdk.
[7] M. R. Guthaus et al., "OpenRAM: An Open-Source Memory Compiler," in IEEE/ACM ICCAD, 2016.
[8] T. Ajayi et al., "Toward an Open-Source Digital Flow: First Learnings from the OpenROAD Project," in ACM/IEEE DAC, 2019.
[9] LibreLane, "LibreLane: ASIC implementation flow," [Online]. Disponível: https://github.com/librelane/librelane.
[10] Accellera Systems Initiative, "Universal Verification Methodology (UVM) 1.1d Class Reference," 2013.
[11] Cadence Design Systems, "Genus Synthesis Solution User Guide."
[12] Cadence Design Systems, "Modus DFT Software Solution User Guide."
[13] AMD, "Kria KR260 Robotics Starter Kit," [Online]. Disponível: https://www.amd.com/en/products/system-on-modules/kria/k26/kr260-robotics-starter-kit.html.
[14] L. A. da Silva, "Projeto e implementação de um módulo ISFFT para modulação OTFS em fluxos de FPGA e ASIC," relatório do Estudo Dirigido 1, Programa CI Digital, 2026.